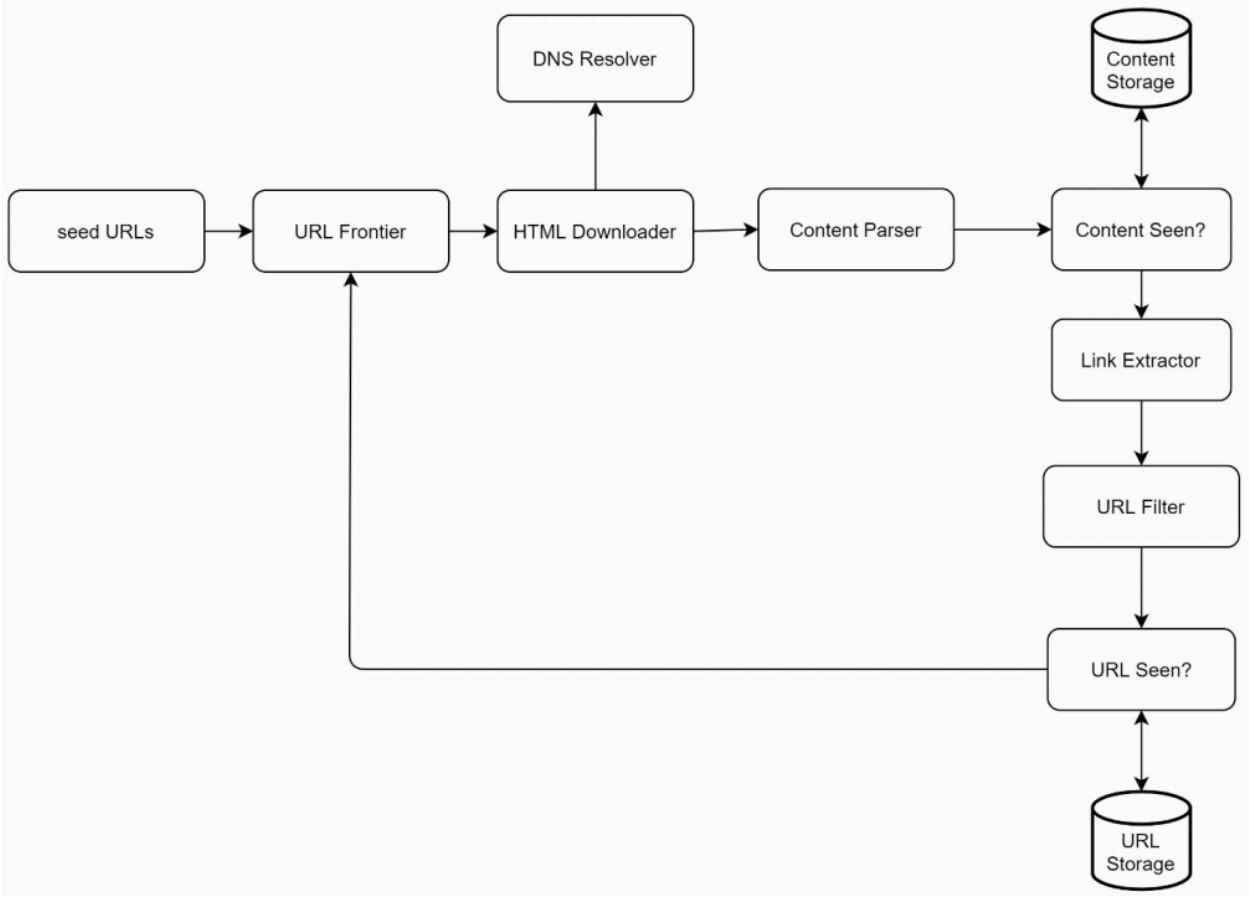
Web Crawler - Step 2: High-Level Design

1. Overview

Once requirements are clear, we design the **high-level architecture**.



Approach:

- Break system into **independent components**
- Understand each component

- Then connect them into a workflow

2. Core Components

Seed URLs

Starting point of crawler

- Initial set of URLs to begin crawling

Selection Strategies:

- **Domain-based**
  - Example: university website
- **Geographical (locality-based)**
  - Different countries → different popular sites
- **Topic-based**
  - Categories:
    - Shopping
    - Sports
    - Healthcare

✔ Good seed selection → better crawl coverage

URL Frontier

Stores URLs to be crawled

- Acts like a **queue (FIFO)**

Responsibilities:

- Maintain:
  - URLs to be downloaded
- Avoid duplicates (with help of URL Seen)

HTML Downloader

Downloads web pages from internet

- Takes URLs from **URL Frontier**
- Fetches HTML content

DNS Resolver

Converts URL → IP address

Example:

```
None
www.wikipedia.org → 198.35.26.96
```

- Required before making network request

Content Parser

Parses and validates HTML

- Handles:
    - Malformed HTML
    - Data cleaning

Separate component to:

- Avoid slowing down crawler

Content Seen?

Detects duplicate content

- ~29% web pages are duplicates

Approach:

- Compare **hash values** instead of raw content

✔ Faster than character-by-character comparison
✔ Reduces storage + processing

Content Storage

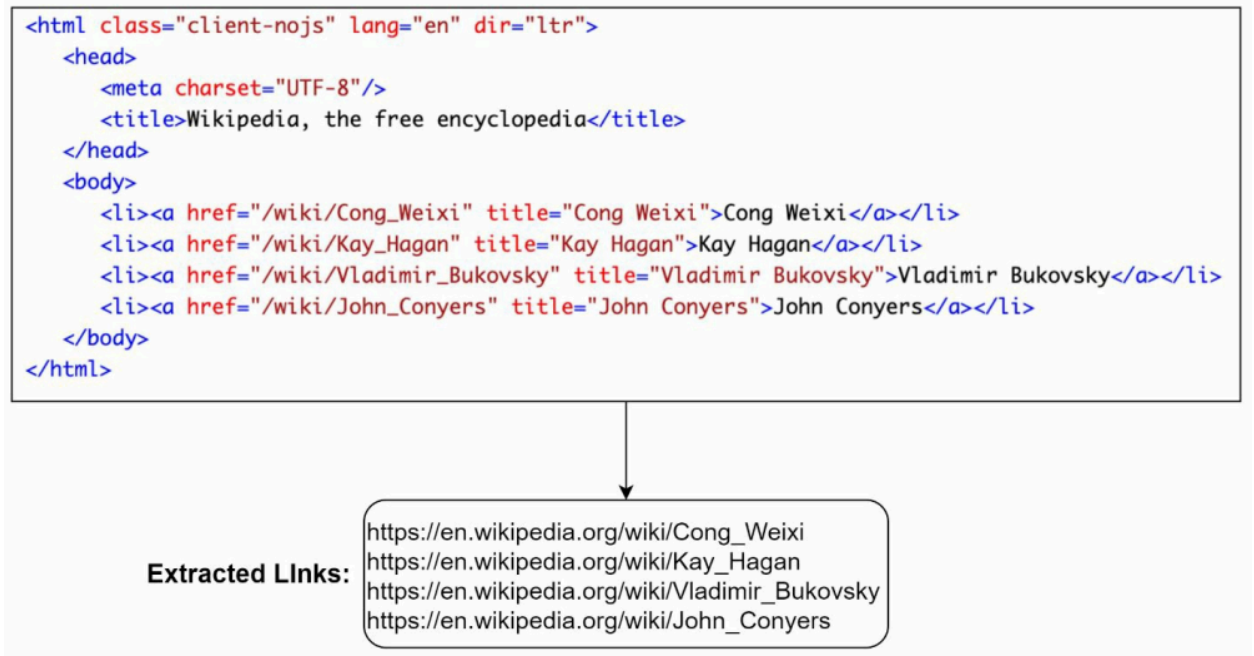
Stores crawled data

Storage Strategy:

- **Disk**
  - Stores majority (large data)
- **Memory**
  - Stores frequently accessed content

URL Extractor

Extracts links from HTML



Responsibilities:

- Parse HTML
- Extract URLs
- Convert:
    - Relative → Absolute URLs

Example:

```
None
/wiki/page → https://en.wikipedia.org/wiki/page
```

URL Filter

Filters unwanted URLs

Removes:

- Unsupported file types
- Error links
- Blacklisted URLs

URL Seen?

Tracks visited URLs

Purpose:

- Prevent duplicate crawling
- Avoid infinite loops

Implementation:

- Bloom Filter
- Hash Table

URL Storage

Stores already visited URLs

- Persistent storage of crawled URLs

3. Key Design Insights

Separation of Concerns

- Each component handles a **specific responsibility**
- Improves:
    - Scalability
    - Maintainability

Efficiency Optimization

- Duplicate detection (Content Seen)
- URL deduplication (URL Seen)
- Filtering unwanted data

Scalability

- Components can be:
    - Distributed
    - Parallelized

4. Key Takeaways

- Web crawler = pipeline of multiple components

- URL Frontier + Downloader drive crawling
- Deduplication is critical for performance
- Storage must handle **massive scale**